

The Ledger Valuation Stack

Pricing, PnL Explain, and Temporal Orchestration

Companion Document to the Ledger Framework Specification

R. Delloye

March 2026 — v0.0

Abstract

The Ledger framework [1] defines portfolio value as $V_t = \sum_u \text{own}(u) \cdot P_t(u)$ and proves that PnL is path-independent: it depends only on the initial and final snapshots, not on intermediate trading activity. The framework deliberately treats P_t as an external input—“a function from units to prices in the reference currency”—without specifying how that function is computed, validated, or delivered.

This companion document fills the gap. It defines the *ValuationRecord*—a structured object that enriches the scalar price with Greeks, model provenance, market data lineage, and a quality flag—and shows that the path-independence theorem is preserved because the scalar `dirty_price` field is precisely the $P_t(u)$ of the main framework. It introduces the *Pricing DAG*, the directed acyclic graph of dependencies between units and market data observables, and specifies how Temporal.io workflows implement reactive, DAG-driven pricing: one workflow per unit, upstream signals triggering downstream re-pricing, heterogeneous compute budgets, and stale-price policies. It formalises PnL explain—the Taylor-expansion verification gate that every new valuation must pass before publication—connecting the multivariate polynomial analysis of [2] to the operational pipeline. Finally, it traces a complete valuation cycle through a three-instrument portfolio (equity spot, vanilla call, structured note embedding the call), showing every workflow signal, pricer invocation, PnL explain check, and publication step with concrete numbers.

The document is self-contained and independently readable. It references the Ledger specification and the PnL explain paper by citation, not by import.

Contents

1	Introduction	3
1.1	Relationship to the Ledger Specification	3
1.2	Relationship to the PnL Explain Paper	3
1.3	Document Roadmap	4
2	The Valuation Record	5
2.1	Quality Semantics	5
2.2	Dirty Price, Clean Price, and Accrued Interest	6
3	Greeks by Instrument Class	7
3.1	Equity Greeks	7
3.2	Vanilla Option Greeks	7
3.3	Exotic Option Greeks	7
3.4	Interest Rate Swap Greeks	7
3.5	Bond Greeks	8
3.6	Futures Greeks	8
3.7	Cash Greeks	8

4	The Pricing DAG	9
4.1	Definition	9
4.2	Construction from the Unit Store	9
4.3	DAG Topology by Instrument Class	9
4.4	Example: Three-Instrument Portfolio	10
4.5	DAG Mutation	10
5	Temporal Workflow Architecture for Pricing	11
5.1	Design Principle: One Workflow per Unit	11
5.2	Cadence and Heterogeneous Compute	11
5.3	DAG Signalling	11
5.4	Workflow Pseudocode	12
5.5	Stale-Price Policy	12
5.6	Timeout Configuration	13
5.7	Task Queue Architecture	13
6	PnL Explain as Verification Gate	14
6.1	Mathematical Foundation	14
6.2	The PnL Explain Function	14
6.3	Reconciling Ex Ante and Ex Post	15
6.4	Failure Handling	15
7	The Valuation Store	17
7.1	Schema	17
7.2	Consumers	17
7.3	Point-in-Time Queries and Time Travel	17
8	Edge Cases	18
8.1	Market Data Gap	18
8.2	Model Failure	18
8.3	DAG Mutation Mid-Cycle	18
8.4	Clock Skew	18
8.5	Replay Safety	18
9	Worked Example: Complete Valuation Cycle	19
9.1	Setup	19
9.2	Market Data Arrives (14:30:00 UTC)	19
9.3	Step 1: NVDA Spot Workflow	19
9.4	Step 2: NVDA Call Workflow	19
9.5	Step 3: Structured Note Workflow	21
9.6	Cycle Summary	21
9.7	Portfolio Valuation	21
10	Interaction Summary	22
11	Conclusion and Open Problems	23
11.1	Open Problems	23

1 Introduction

The Ledger framework [1] establishes six properties by construction: atomicity, conservation, determinism, state-sufficiency, lifecycle value invariance, and time travel. Portfolio valuation appears in [1, §4] as:

$$V_t = \sum_{u \in \mathcal{U}} w_t(u) \cdot P_t(u)$$

where $w_t(u)$ is the wallet balance (or, in the generalised position model, $\text{own}(u)$) and $P_t(u)$ is the price of unit u at time t in the reference currency. The path-independence theorem [1, §4.3] proves that $\text{PnL} = V_{t_1} - V_{t_0}$ regardless of intermediate trading activity.

The framework is deliberately agnostic to the pricing methodology: “ P_t is an *external input* to the framework, not computed by it” [1, §4.1]. This is the correct architectural boundary—the ledger is a settlement and position system, not a pricing engine. But the boundary creates three unanswered questions:

1. **What does $P_t(u)$ return?** In practice, a price is not a scalar. A pricing computation produces sensitivities (Greeks), model provenance, market data lineage, compute time, and a quality assessment. How does this structured output interact with the scalar P_t of the PnL theorem?
2. **How are prices computed in dependency order?** An NVDA vanilla call depends on NVDA spot, the NVDA implied volatility surface, and the USD risk-free rate curve. A structured note embedding the call depends on the call’s valuation. How is this dependency graph defined, enforced, and executed?
3. **How are new prices validated before they enter the ledger’s valuation store?** The PnL explain analysis [2] provides the mathematical foundation—Taylor expansion of price changes in terms of Greeks and market data moves—but does not specify the operational pipeline: when does the check run, what happens on failure, how does the system recover?

This document answers all three.

1.1 Relationship to the Ledger Specification

This document is a companion to, not a revision of, the Ledger framework [1]. It introduces no new primitives (no new move types, no new wallet structures, no changes to the conservation law). It specifies the *valuation layer* that produces the P_t function consumed by the ledger’s PnL computation. All definitions, notation, and conventions follow [1] unless stated otherwise.

1.2 Relationship to the PnL Explain Paper

The PnL explain paper [2] establishes the mathematical foundation: for a pricing function P approximated as a polynomial of degree three in the market data vector x , the price change decomposes as:

$$P(x_2) - P(x_1) = \left\langle \frac{\Delta_{x_2} + \Delta_{x_1}}{2} \middle| x_2 - x_1 \right\rangle - \frac{1}{12} \langle x_2 - x_1 | (\Gamma_{x_2} - \Gamma_{x_1}) | x_2 - x_1 \rangle$$

where $\Delta_x = \nabla P(x)$ is the gradient (delta vector) and $\Gamma_x = \nabla^2 P(x)$ is the Hessian (gamma matrix). The paper distinguishes ex ante PnL (using only information at x_1) from ex post PnL (using information at both x_1 and x_2). This document operationalises those results as a verification gate in the pricing pipeline.

1.3 Document Roadmap

Section 2 defines the ValuationRecord. Section 3 specifies the Greeks structure by instrument class. Section 4 introduces the Pricing DAG. Section 5 specifies the Temporal workflow architecture for reactive pricing. Section 6 formalises PnL explain as a verification gate. Section 7 describes the valuation store and its interaction with the ledger. Section 8 catalogues edge cases. Section 9 traces a complete valuation cycle with concrete numbers. Section 11 identifies open problems.

2 The Valuation Record

The Ledger specification defines $P_t : \mathcal{U} \rightarrow \mathbb{R}$ as a scalar function. In practice, the output of a pricing computation is richer. We define the structured output and show that the scalar theorem is preserved.

Definition 2.1 (Valuation Record). *A ValuationRecord for unit u at time t is a tuple:*

```
ValuationRecord:
  unit_id:      UnitId
  timestamp:    Timestamp
  dirty_price:  Decimal      -- full market value including accrual
  clean_price:  Decimal      -- quoted price net of accrual
  accrued:     Decimal      -- dirty_price - clean_price
  greeks:       Greeks       -- instrument-specific sensitivity vector
  model_id:     String       -- which pricer produced this
  market_data_snap: SnapshotId -- which market data was used
  compute_ms:   Int          -- how long pricing took
  quality:      Quality       -- FIRM / INDICATIVE / STALE / FAILED
```

The fields serve distinct consumers:

- **dirty_price**: the scalar consumed by the ledger’s PnL computation ($V_t = \sum \text{own}(u) \cdot \text{dirty_price}_t(u)$).
- **clean_price, accrued**: reporting fields for bond and loan instruments where market convention quotes a clean price.
- **greeks**: the sensitivity vector consumed by PnL explain (Section 6) and by the risk pipeline.
- **model_id, market_data_snap**: provenance fields for audit and model risk management.
- **compute_ms**: operational telemetry for capacity planning.
- **quality**: a flag consumed by the risk pipeline and by the obligation-liveness executor [1, §14.7] when deciding whether to act on a valuation.

Principle 2.2 (Scalar Compatibility). *The ValuationRecord enriches but does not replace the scalar price function. The **dirty_price** field IS the $P_t(u)$ of [1, §4.1]. The path-independence theorem [1, §4.3] is preserved:*

$$\text{PnL} = V_{t_1} - V_{t_0} = \sum_u \text{own}(u) \cdot [\text{dirty_price}_{t_1}(u) - \text{dirty_price}_{t_0}(u)]$$

The remaining fields are metadata that enriches the record without altering the theorem. No downstream consumer that depends on the scalar P_t needs modification.

2.1 Quality Semantics

The quality flag has four values with precise operational semantics:

Quality	Semantics
FIRM	Price computed from live market data within the unit's cadence window; PnL explain passed. This is the normal state.
INDICATIVE	Price computed but PnL explain tolerance exceeded, or market data is from a secondary source (e.g., composite quote instead of primary exchange). Risk pipeline treats with elevated scrutiny.
STALE	Price is the last-known-good value; the pricer has not produced a fresh price within its cadence window. The <code>timestamp</code> field shows when the price was last computed, not the current time.
FAILED	The pricer failed to produce any price. No <code>dirty_price</code> is available; downstream consumers use the previous FIRM or INDICATIVE price with STALE quality.

The quality flag is consumed by two downstream systems. The risk pipeline uses it for limit monitoring: a position valued at STALE triggers a staleness alert and may receive a prudential haircut. The obligation-liveness executor [1, §14.7] uses it for margin calculations: a CSA VM call computed on STALE valuations is flagged for manual review before dispatch.

2.2 Dirty Price, Clean Price, and Accrued Interest

For instruments with accrual mechanics (bonds, loans, swaps with accrued interest), the `dirty_price` is the economically complete price:

$$\text{dirty_price} = \text{clean_price} + \text{accrued}$$

The clean price is the market-quoted price excluding interest that has accrued since the last coupon date. The accrued interest is computed from the unit state [1, §7], which records the last coupon date, the coupon rate, and the day-count convention. This is consistent with [1, §5.3]: “the bond’s price function $P_t(u)$ returns the *dirty* price.”

For instruments without accrual (equities, cash, most derivatives), `clean_price` = `dirty_price` and `accrued` = 0.

3 Greeks by Instrument Class

The `greeks` field of the `ValuationRecord` is a tagged union whose active variant depends on the instrument class. This reflects a fundamental reality: different instruments have different risk factors, and a one-size-fits-all sensitivity vector would be either incomplete or wasteful.

Definition 3.1 (Greeks). *The Greeks structure is an instrument-class-specific sensitivity vector:*

```
Greeks = EquityGreeks / OptionGreeks / ExoticOptionGreeks
        / IRSGreeks / BondGreeks / FuturesGreeks / CashGreeks
```

3.1 Equity Greeks

```
EquityGreeks:
  delta:    Decimal = 1.0    -- dP/dS; always 1.0 for a spot position
```

An equity spot position has delta of exactly 1.0 by definition: a \$1 move in the spot price changes the position value by \$1 per share. All higher-order sensitivities are zero.

3.2 Vanilla Option Greeks

```
OptionGreeks:
  delta:    Decimal    -- dP/dS
  gamma:    Decimal    -- d^2P/dS^2
  vega:     Decimal    -- dP/d(sigma)
  theta:    Decimal    -- dP/dt (per calendar day)
  rho:      Decimal    -- dP/dr
```

These are the five first- and second-order sensitivities of the Black-Scholes-Merton model. For PnL explain purposes, the Taylor expansion uses delta and gamma with respect to the underlying price, vega with respect to implied volatility, theta with respect to time, and rho with respect to the risk-free rate.

3.3 Exotic Option Greeks

```
ExoticOptionGreeks:
  delta:    Decimal    -- dP/dS
  gamma:    Decimal    -- d^2P/dS^2
  vega:     Decimal    -- dP/d(sigma)
  theta:    Decimal    -- dP/dt
  rho:      Decimal    -- dP/dr
  vanna:    Decimal    -- d^2P/(dS d(sigma))
  volga:    Decimal    -- d^2P/d(sigma)^2
  charm:    Decimal    -- d^2P/(dS dt)
```

Exotic options (barriers, Asians, lookbacks) require cross-sensitivities for accurate PnL explain. Vanna captures the sensitivity of delta to volatility, volga captures the convexity of price with respect to volatility, and charm captures the time-decay of delta. These terms appear in the second-order and cross-term components of the multivariate Taylor expansion [2].

3.4 Interest Rate Swap Greeks

```
IRSGreeks:
  dv01:      Decimal    -- dollar value of one basis point
  convexity:  Decimal    -- second-order rate sensitivity
  key_rate_durations: Map<Tenor, Decimal> -- per-tenor DV01
```

For an IRS, the risk factor is the yield curve, not a single spot price. DV01 is the change in present value for a one-basis-point parallel shift; key-rate durations decompose this sensitivity across tenors (1y, 2y, 5y, 10y, 30y, etc.). The PnL explain for an IRS uses:

$$\text{explained PnL} \approx \sum_{\tau} \text{KRD}_{\tau} \cdot \Delta r_{\tau} + \frac{1}{2} \cdot \text{convexity} \cdot (\Delta r_{\text{par}})^2$$

where Δr_{τ} is the change in the zero rate at tenor τ .

3.5 Bond Greeks

```
BondGreeks:
  modified_duration: Decimal    -- -(1/P) (dP/dy)
  convexity:         Decimal    -- (1/P) (d^2P/dy^2)
  dv01:              Decimal    -- dollar value of one basis point
```

Bond Greeks are expressed in yield-to-maturity space. Modified duration and convexity drive the PnL explain:

$$\frac{\Delta P}{P} \approx -D_{\text{mod}} \cdot \Delta y + \frac{1}{2} \cdot C \cdot (\Delta y)^2$$

3.6 Futures Greeks

```
FuturesGreeks:
  delta:              Decimal = 1.0    -- by definition
  accumulated_vm:     Decimal          -- variation margin to date
```

A futures contract has delta of 1.0 to its settlement price. The `accumulated_vm` field records the cumulative variation margin, connecting to the daily settlement workflow [1, §14.4.1].

3.7 Cash Greeks

```
CashGreeks:
  -- empty: no sensitivities for the reference currency
```

The reference currency (e.g., USD) has $P_t(\text{USD}) = 1$ by definition. Foreign currencies have an FX delta relative to the reference currency, modelled as an EquityGreeks with delta equal to the FX rate.

4 The Pricing DAG

Pricing is not a flat operation over independent instruments. The price of an NVDA vanilla call depends on NVDA spot, the NVDA implied volatility surface, and the USD risk-free rate curve. A structured note embedding the call depends on the call’s valuation. These dependencies form a directed acyclic graph—the *Pricing DAG*.

4.1 Definition

Definition 4.1 (Pricing DAG). *The Pricing DAG is a directed acyclic graph $G = (N, E)$ where:*

- $N = N_U \cup N_M$: *nodes are either unit nodes ($N_U \subseteq \mathcal{U}$, the units requiring valuation) or market data nodes (N_M , observable data feeds such as spot prices, yield curves, volatility surfaces).*
- $E \subseteq N \times N$: *a directed edge $(a, b) \in E$ means “b requires a fresh price or data point from a before it can be priced.”*

Market data nodes are leaf nodes (no incoming edges): they receive prices from external data feeds. Equity spot positions are also near-leaf: they depend on a single market data node (the exchange price feed). Derivatives and structured products are interior or root nodes with dependencies on both market data and other unit nodes.

4.2 Construction from the Unit Store

The DAG is not manually specified. It is constructed from the unit definitions in the Unit Store [1, §3]:

1. Each unit’s smart contract declares its *pricing observables*—the market data nodes it requires. For an NVDA call, the contract declares: NVDA spot, NVDA vol surface, USD rate curve.
2. Each unit’s smart contract may declare *unit dependencies*—other units whose valuations it requires. For a structured note embedding an NVDA call, the contract declares: the specific NVDA call unit.
3. The DAG is the transitive closure of these declarations.

Invariant 4.2 (Acyclicity). *The Pricing DAG is acyclic. This invariant is enforced at unit registration [1, §3.4]: the Unit Store rejects a unit whose pricing dependencies would create a cycle.*

Acyclicity ensures that a topological ordering exists, guaranteeing that every unit can be priced after all its dependencies. In practice, cycles are rare in financial instrument pricing (a call on NVDA does not have NVDA depending on the call), but the invariant prevents pathological configurations such as circular structured notes.

4.3 DAG Topology by Instrument Class

Instrument	Upstream dependencies	DAG depth
Cash (USD)	None	0 (by definition)
Equity spot (NVDA)	NVDA exchange price feed	1
Vanilla option (NVDA call)	NVDA spot, NVDA vol surface, USD rate curve	2
Exotic option (barrier)	NVDA spot, NVDA vol surface, USD rate curve	2
IRS	USD yield curve (deposits, futures, swaps)	2
Bond	Issuer yield curve, USD rate curve	2
Structured note	Embedded option valuation, issuer credit curve	3+

4.4 Example: Three-Instrument Portfolio

Figure 1 illustrates the Pricing DAG for the worked example in Section 9: a portfolio containing NVDA spot, an NVDA Jun 2026 900 Call, and a structured note embedding the call.

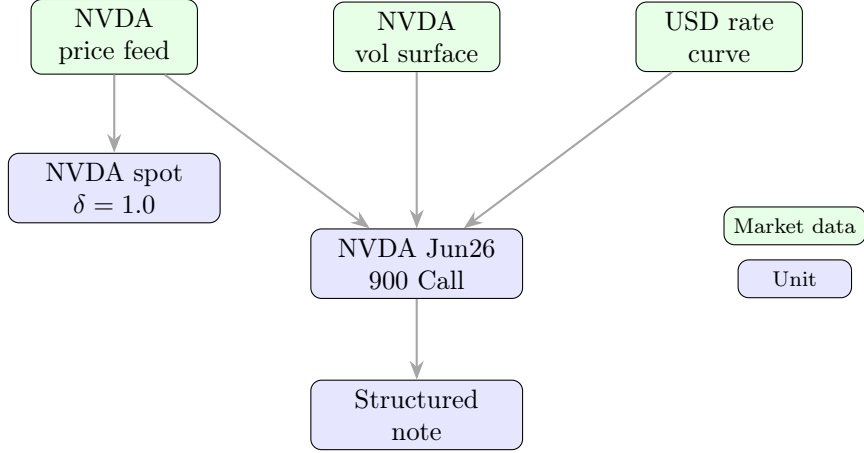


Figure 1: Pricing DAG for the three-instrument portfolio. Green nodes are market data feeds (leaf nodes); blue nodes are units requiring valuation. Edges point from dependency to dependent. NVDA spot depends only on its price feed. The NVDA call depends on NVDA spot (via the price feed), the vol surface, and the rate curve. The structured note depends on the call's valuation.

4.5 DAG Mutation

The Pricing DAG is not static. It changes when:

- A new unit is registered [1, §3.4]: the unit's pricing dependencies create new nodes and edges.
- A unit matures or terminates: the unit node and its edges are removed (the unit's ValuationRecord receives a final price at termination, and no further pricing cycles are required).
- A market data source is switched (e.g., from primary exchange feed to composite): the market data node's identity changes, but the topology is preserved.

DAG mutations take effect at the next pricing cycle. The acyclicity invariant is re-checked at every unit registration. In-flight pricing cycles are not affected: a new unit added mid-cycle will be priced on the next cycle, not retroactively inserted into the current one.

5 Temporal Workflow Architecture for Pricing

The Ledger specification uses Temporal.io as its execution engine [1, §14]: lifecycle events, settlement, and obligation monitoring are orchestrated as Temporal workflows. The valuation stack extends this architecture with pricing-specific workflows that implement the reactive, DAG-driven pricing pattern.

5.1 Design Principle: One Workflow per Unit

Each unit in the Pricing DAG has a dedicated `PricingWorkflow(unit_id)` that:

1. waits for all upstream dependencies to signal fresh prices,
2. retrieves market data,
3. invokes the pricer activity,
4. runs PnL explain verification,
5. publishes the `ValuationRecord`.

This is a one-to-one mapping: every unit node in the Pricing DAG corresponds to exactly one long-running Temporal workflow. The workflow is instantiated at unit registration and completed at maturity or termination, mirroring the lifecycle workflows of [1, §14.4].

5.2 Cadence and Heterogeneous Compute

Different instruments require different pricing frequencies and have different compute budgets:

Instrument class	Cadence	Compute budget	Pricer type
Equity spot	1–5s	< 1ms	Trivial (pass-through)
Vanilla option	10–30s	1–10ms	Analytical (Black-Scholes)
Exotic option	1–5min	1–30s	Monte Carlo / PDE
IRS	30s–2min	10–100ms	Curve bootstrapping
Bond	30s–2min	1–10ms	Yield-to-maturity
Structured note	5–15min	1–60s	Composite (depends on embedded)

The cadence is a configurable parameter on each `PricingWorkflow`, not a global constant. Liquid equities may be priced every second; illiquid OTC exotics may be priced every 15 minutes or only on demand.

5.3 DAG Signalling

The reactive pricing pattern is implemented through Temporal signals:

1. When a leaf node (e.g., NVDA price feed) produces a new data point, it signals all downstream unit workflows that depend on it.
2. When a unit workflow produces a new `ValuationRecord`, it signals all downstream unit workflows that depend on it.
3. A downstream workflow maintains a freshness map: `Map<DependencyId, Timestamp>`. When a signal arrives, the map is updated. The workflow checks: “do I have fresh prices for ALL my upstream nodes?” If yes, fire pricing. If not, wait.

This is a data-flow / reactive pattern. It has two key properties:

- **No redundant computation.** A workflow does not re-price until all upstream dependencies have refreshed. If NVDA spot updates but the vol surface has not, the call’s workflow waits.
- **Automatic cascade.** When the NVDA call produces a new `ValuationRecord`, it signals the structured note workflow, which then re-prices. The cascade propagates through the DAG without explicit orchestration.

5.4 Workflow Pseudocode

```
PricingWorkflow(unit_id, cadence, dependencies):
    prev_record = None
    freshness = {dep: None for dep in dependencies}
    upstream_ch = workflow.GetSignalChannel("upstream_ready")

    while unit_state(unit_id).lifecycle_stage == ACTIVE:
        -- Wait for upstream signals or cadence timer
        selector = workflow.NewSelector()
        selector.AddReceive(upstream_ch, func(signal):
            freshness[signal.dep_id] = signal.timestamp
        )
        selector.AddTimer(cadence, func(): pass)
        selector.Select()

        -- Check all dependencies are fresh
        if not all_fresh(freshness, cadence):
            continue

        -- Retrieve market data
        mkt_snap = await RetrieveMarketData(unit_id, dependencies)

        -- Invoke pricer
        raw_record = await InvokePricer(unit_id, mkt_snap)

        -- PnL explain gate
        if prev_record is not None:
            explain = await PnLExplain(prev_record, raw_record,
                                      mkt_snap.delta_from(prev_snap))
            if explain.status == FAIL:
                await Quarantine(raw_record, explain)
                raw_record = await RetryPricing(unit_id, mkt_snap)
                explain = await PnLExplain(prev_record, raw_record,
                                          mkt_snap.delta_from(prev_snap))
            if explain.status == FAIL:
                await AlertRiskTeam(unit_id, explain)
                raw_record.quality = STALE
                raw_record = prev_record.with_quality(STALE)

        -- Publish
        await PublishValuationRecord(raw_record)
        prev_record = raw_record

        -- Signal downstream dependents
        for dep in downstream_units(unit_id):
            workflow.SignalExternalWorkflow(
                dep.workflow_id, "upstream_ready",
                {dep_id: unit_id, timestamp: raw_record.timestamp})
```

5.5 Stale-Price Policy

A pricer may fail to produce a fresh price within its cadence window for several reasons: market data gap, model failure, compute timeout, or upstream dependency staleness. The system applies a two-tier policy:

1. **Within cadence window:** the workflow waits. No action is taken.

2. **Beyond cadence window (up to $3\times$ cadence)**: the last-known-good price is retained with `quality = STALE`. The staleness flag propagates to downstream dependents: if the NVDA call’s workflow sees that NVDA spot is STALE, it either re-prices with stale input (noting the staleness in its own record) or waits, depending on configuration.
3. **Beyond $3\times$ cadence**: an alert is raised to the risk team via the obligation-liveness framework [1, §14.7]. A pricing failure beyond $3\times$ cadence is treated as an event-triggered obligation with a compensation action of “use last-known-good with quality STALE and escalate.”

5.6 Timeout Configuration

Each pricer activity has Temporal activity options matching its compute budget:

```

PricerActivityOptions(instrument_class):
    match instrument_class:
        case EQUITY_SPOT:
            StartToCloseTimeout: 5s
            ScheduleToCloseTimeout: 30s
        case VANILLA_OPTION:
            StartToCloseTimeout: 30s
            ScheduleToCloseTimeout: 2min
        case EXOTIC_OPTION:
            StartToCloseTimeout: 5min
            ScheduleToCloseTimeout: 15min
        case IRS | BOND:
            StartToCloseTimeout: 2min
            ScheduleToCloseTimeout: 5min
    RetryPolicy:
        InitialInterval: 1s
        BackoffCoefficient: 2.0
        MaximumAttempts: 3
        NonRetryableErrors: [ModelNotFound, UnitNotActive]

```

The retry policy mirrors the executor activity options [1, §14.2]: transient failures (network, compute resource) are retried; logic errors (model not found, unit no longer active) are non-retryable.

5.7 Task Queue Architecture

Pricing workflows and activities are distributed across dedicated Temporal task queues to isolate compute-intensive pricers from fast, latency-sensitive ones:

Task queue	Instruments	Worker profile
<code>pricing-fast</code>	Equity spot, FX, cash	Low-latency, high-throughput
<code>pricing-analytical</code>	Vanilla options, bonds, IRS	Medium compute
<code>pricing-mc</code>	Exotic options, structured notes	GPU-enabled, high memory
<code>pricing-workflow</code>	All PricingWorkflow orchestrators	Lightweight, many instances

Workflow code runs on `pricing-workflow` workers; pricer activities are dispatched to the appropriate compute queue based on the instrument class. This separation allows scaling expensive Monte Carlo workers independently of lightweight equity spot workers.

6 PnL Explain as Verification Gate

Every new ValuationRecord must pass through PnL explain before publication. This section formalises the gate, connecting the mathematical foundation of [2] to the operational pipeline.

6.1 Mathematical Foundation

The PnL explain paper [2] establishes three results for a pricing function P treated as a polynomial of degree three in market data vector $x \in \mathbb{R}^n$:

Ex post (exact for cubic P).

$$P(x_2) - P(x_1) = \left\langle \frac{\Delta_{x_2} + \Delta_{x_1}}{2} \middle| x_2 - x_1 \right\rangle - \frac{1}{12} \langle x_2 - x_1 | (\Gamma_{x_2} - \Gamma_{x_1}) | x_2 - x_1 \rangle \quad (1)$$

Ex ante (second-order approximation using only x_1 information).

$$P(x_2) - P(x_1) \approx \langle \Delta_{x_1} | x_2 - x_1 \rangle + \frac{1}{2} \langle x_2 - x_1 | \Gamma_{x_1} | x_2 - x_1 \rangle \quad (2)$$

Decomposition.

$$P(x_2) - P(x_1) = \underbrace{\langle \Delta_{x_1} | x_2 - x_1 \rangle}_{\text{1st order}} + \underbrace{\left\langle \frac{\Delta_{x_2} - \Delta_{x_1}}{2} \middle| x_2 - x_1 \right\rangle}_{\text{2nd order}} - \underbrace{\frac{1}{12} \langle x_2 - x_1 | (\Gamma_{x_2} - \Gamma_{x_1}) | x_2 - x_1 \rangle}_{\text{3rd order}} \quad (3)$$

In the operational setting, the market data vector x comprises the risk factors relevant to the instrument: spot price, implied volatility, risk-free rate, yield curve tenors, time. The gradient $\Delta_x = \nabla P(x)$ is the instrument's Greek vector, and the Hessian $\Gamma_x = \nabla^2 P(x)$ is the gamma matrix.

6.2 The PnL Explain Function

Definition 6.1 (PnL Explain). *The PnL explain function takes two consecutive ValuationRecords, the market data delta, and any intervening cashflows, and produces an ExplainResult:*

```
pnl_explain(
  prev:      ValuationRecord,
  curr:      ValuationRecord,
  market_moves: MarketDataDelta,
  cashflows: List[Move]
) -> ExplainResult
```

The computation proceeds as follows:

```
total_pnl = curr.dirty_price - prev.dirty_price

-- Taylor expansion using prev Greeks (ex ante)
explained = (
  prev.greeks.delta * market_moves.spot_change
+ 0.5 * prev.greeks.gamma * market_moves.spot_change^2
+ prev.greeks.vega * market_moves.vol_change
+ prev.greeks.theta * dt
+ prev.greeks.rho * market_moves.rate_change
+ sum(cf.amount for cf in cashflows)
)
```

```

unexplained = total_pnl - explained

if abs(unexplained) < tolerance(unit_id):
    return ExplainResult(status=PASS,
        unexplained=unexplained,
        breakdown={delta=..., gamma=..., vega=...,
            theta=..., rho=..., cashflow=...})
else:
    return ExplainResult(status=FAIL,
        unexplained=unexplained,
        breakdown={...})

```

The tolerance is instrument-specific and scales with the notional. Typical thresholds:

Instrument class	Tolerance (per unit notional)
Equity spot	\$0.01 (essentially zero: explain is exact)
Vanilla option	1% of option premium
Exotic option	2% of option premium
IRS	0.5 basis points of notional
Bond	0.1% of face value

6.3 Reconciling Ex Ante and Ex Post

In the operational pipeline, the PnL explain gate uses the *ex ante* formula (Equation 2): at the time of pricing, only the Greeks from the previous valuation (`prev.greeks`) and the market data moves are available. The approximated delta at the new point, $\tilde{\Delta}_{x_2} = \Gamma_{x_1}|x_2 - x_1\rangle + |\Delta_{x_1}\rangle$ [2, §4.1], is the delta used for intra-day risk management.

After the new valuation is published (with its own Greeks), the *ex post* decomposition (Equation 3) becomes available. The ex post result is stored alongside the ValuationRecord for end-of-day PnL reporting and model risk analysis. The difference between ex ante and ex post unexplained PnL is a diagnostic for model adequacy: if the difference is consistently large, the pricing model’s Taylor expansion is a poor approximation (e.g., the option is deep out-of-the-money and gamma is rapidly changing).

6.4 Failure Handling

When PnL explain FAILS:

1. **Quarantine.** The new ValuationRecord is not published to the main valuation store. It is held in a quarantine area indexed by (`unit_id`, `timestamp`).
2. **Alert.** A pricing alert is raised via the obligation-liveness framework [1, §14.7], creating an event-triggered obligation with:
 - Deadline: $3\times$ the unit’s pricing cadence.
 - Discharge predicate: a subsequent PnL explain passes.
 - Compensation action: retain last-known-good price with STALE quality.
3. **Retry with refreshed market data.** The PricingWorkflow re-invokes the market data retrieval activity and the pricer activity. Market data gaps or stale quotes may have caused the failure.
4. **Retry with alternative model.** If the primary model (e.g., Monte Carlo) fails PnL explain, a fallback model (e.g., analytical Black-Scholes for an option near expiry) is attempted. The `model_id` field records which pricer produced the result.

5. **Manual override.** If all automated retries fail, a human trader or risk manager can provide a manual price. The `ValuationRecord` is published with `quality = INDICATIVE` and `model_id = 'MANUAL_OVERRIDE'`.
6. **Last resort: STALE.** If no valid price can be produced, the last-known-good `ValuationRecord` is retained with `quality = STALE`. The failure is journaled in the event log [1, §8.2] with full diagnostic information.

This escalation ladder ensures that the valuation store always contains a price for every active unit, even in degraded conditions. The quality flag makes the degradation visible to all downstream consumers.

7 The Valuation Store

ValuationRecords are stored in a *valuation store* that is conceptually separate from the move stream [1, §8]. The move stream records economic activity (moves and state transitions); the valuation store records price observations.

7.1 Schema

The valuation store is indexed by (`unit_id`, `timestamp`):

```
ValuationStore:
  primary_key: (unit_id, timestamp)
  value:       ValuationRecord
  indexes:
    - by_unit_id:    for all prices of a unit, ordered by time
    - by_timestamp:  for all prices at a point in time (snapshot)
    - by_quality:    for monitoring STALE and FAILED prices
```

The valuation store is append-only within a pricing cycle: new records are inserted, never updated. Corrections are recorded as new entries with a later timestamp and appropriate metadata (e.g., `model_id` = “CORRECTION”).

7.2 Consumers

Four systems consume the valuation store:

1. **Ledger PnL computation.** The ledger reads `dirty_price` for portfolio valuation: $V_t = \sum \text{own}(u) \cdot \text{dirty_price}_t(u)$. This is the primary consumer. PnL attribution (price PnL vs. flow PnL, per [1, §4.4]) reads both the t_0 and t_1 snapshots.
2. **Obligation-liveness executor.** CSA variation margin depends on portfolio mark-to-market. The executor reads the latest ValuationRecord for each unit under the CSA, computes aggregate MTM, and triggers margin calls [1, §14.4.4]. The quality flag is checked: STALE valuations trigger a margin call with a review flag.
3. **Risk pipeline.** The risk pipeline reads Greeks for limit monitoring (delta limits, gamma limits, DV01 limits, VaR), stress testing (parallel shift, curve twist, vol shock), and regulatory capital calculations. Greeks are consumed from the same ValuationRecord used for PnL, ensuring consistency.
4. **PnL explain.** The PnL explain gate (Section 6) reads the previous ValuationRecord as input. This creates a self-referential loop: the valuation store feeds PnL explain, which gates publication to the valuation store. The loop is well-defined because PnL explain for the first ValuationRecord of a unit is a no-op (there is no previous record to compare against).

7.3 Point-in-Time Queries and Time Travel

All reads are point-in-time. The valuation store supports the same time-travel capability as the move stream [1, §1.2, Property 6]:

- `snapshot_at(t)`: returns the latest ValuationRecord for each unit with `timestamp` $\leq t$. This reconstructs the valuation as it was known at time t .
- `history(unit_id, t0, t1)`: returns all ValuationRecords for a unit in the interval $[t_0, t_1]$, ordered by timestamp. This supports time-series analysis of prices and Greeks.

The distinction between “time travel to what we knew at time t ” and “time travel with today’s corrected data” [1, §1.2] applies to the valuation store as well. The store records what was computed at the time; corrections and restatements are separate entries.

8 Edge Cases

8.1 Market Data Gap

When a market data feed is unavailable (exchange halt, data vendor outage, pre-market hours), the pricer uses the last-known-good data point with the STALE flag propagated through the ValuationRecord. The Pricing DAG’s signalling mechanism handles this naturally: the market data node does not signal downstream dependents until fresh data arrives, so downstream workflows either wait (if configured for strict freshness) or proceed with stale data (if configured for best-effort).

8.2 Model Failure

When a pricer crashes, times out, or produces a numerically invalid result (NaN, Inf, negative price for a non-negative instrument), the PricingWorkflow applies the escalation ladder of Section 6:

1. Retry with same model and refreshed market data.
2. Fallback to simpler model (e.g., analytical Black-Scholes instead of Monte Carlo for an option near expiry, where the analytical model is sufficient).
3. Manual override or STALE retention.

The fallback model hierarchy is configured per instrument class in the Unit Store’s Tier 2 product registry [1, §3.2.2].

8.3 DAG Mutation Mid-Cycle

A new trade booked mid-cycle creates a new unit in the Unit Store. The corresponding PricingWorkflow is instantiated, and the DAG is extended with the new node and edges. The new workflow participates in the next pricing cycle; it does not retroactively affect the current cycle.

Conversely, when a unit terminates (option expiry, bond maturity, IRS termination), the PricingWorkflow completes, the DAG node is removed, and downstream dependents are notified that the dependency no longer exists. If a structured note embeds a terminated option, the note’s pricer must handle the missing dependency (typically by using the option’s terminal value, which is deterministic).

8.4 Clock Skew

Temporal’s server-side timestamps are authoritative for workflow scheduling [1, §14]. The ValuationRecord’s `timestamp` is set by the pricing activity, using the market data snapshot’s timestamp (not the worker’s system clock). This ensures that valuations are anchored to market data time, not compute time.

8.5 Replay Safety

Pricing activities are idempotent: the same market data snapshot and unit state produce the same ValuationRecord. This follows from the purity principle [1, §7.7]: pricing functions are pure functions of their inputs. Temporal’s deterministic replay guarantee [1, §14.1] ensures that a replayed PricingWorkflow invokes the same activities with the same inputs, producing the same outputs.

Remark 8.1 (Monte Carlo and Deterministic Seeds). *Monte Carlo pricers use pseudo-random number generators seeded from the market data snapshot ID and unit ID. This ensures that the same inputs produce the same random paths and hence the same price, satisfying the purity requirement. The seed derivation is: `seed = hash(market_data_snap, unit_id)`.*

9 Worked Example: Complete Valuation Cycle

This section traces a complete valuation cycle through the three-instrument portfolio of Figure 1. All numbers are illustrative.

9.1 Setup

Unit	Description	Cadence
NVDA	NVIDIA Corp. equity spot	5s
NVDA-C-Jun26-900	NVDA Jun 2026 900 Call (European, cash-settled)	30s
SN-NVDA-001	Structured note embedding the call	5min

Previous valuations (at 14:29:30 UTC):

Unit	Dirty price	δ	γ	ν	θ	Quality
NVDA	\$908.00	1.000	—	—	—	FIRM
NVDA-C-Jun26-900	\$68.20	0.571	0.0039	0.298	-0.138	FIRM
SN-NVDA-001	\$102.30	—	—	—	—	FIRM

9.2 Market Data Arrives (14:30:00 UTC)

Fresh market data:

- NVDA spot: \$912.50 (was \$908.00; $\Delta S = +\$4.50$)
- NVDA 30-day implied vol: 42.3% (was 41.8%; $\Delta \sigma = +0.5\%$)
- USD risk-free rate: 5.20% (unchanged; $\Delta r = 0$)

9.3 Step 1: NVDA Spot Workflow

The NVDA price feed signals the NVDA spot workflow. The pricer is trivial: `dirty_price = spot_price`.

Field	Value
<code>dirty_price</code>	\$912.50
<code>delta</code>	1.000
<code>model_id</code>	EQUITY_PASSTHROUGH
<code>compute_ms</code>	0

PnL explain.

$$\text{Total PnL} = 912.50 - 908.00 = +\$4.50$$

$$\text{Explained} = \delta \times \Delta S = 1.0 \times 4.50 = +\$4.50$$

$$\text{Unexplained} = 4.50 - 4.50 = \$0.00 \quad \checkmark \text{ PASS}$$

The ValuationRecord is published with `quality = FIRM`. The workflow signals all downstream dependents, including the NVDA call workflow.

9.4 Step 2: NVDA Call Workflow

The NVDA call workflow receives three signals: NVDA spot (from Step 1), NVDA vol surface (from market data), and USD rate curve (from market data). All upstream dependencies are fresh. The pricer invokes Black-Scholes with:

Parameter	Value
S (spot)	\$912.50
K (strike)	\$900.00
σ (implied vol)	42.3%
r (risk-free rate)	5.20%
T (time to expiry)	0.23 years

Black-Scholes output:

Field	Value
dirty_price	\$72.85
δ	0.583
γ	0.0041
ν (vega)	0.312
θ	-0.145
ρ	0.092
model_id	BSM_ANALYTICAL
compute_ms	0.6

PnL explain.

$$\text{Total PnL} = 72.85 - 68.20 = +\$4.65$$

Taylor expansion using previous Greeks ($\delta_{x_1} = 0.571$, $\gamma_{x_1} = 0.0039$, $\nu_{x_1} = 0.298$, $\theta_{x_1} = -0.138$):

$$\text{Delta term} = 0.571 \times 4.50 = +\$2.570$$

$$\text{Gamma term} = \frac{1}{2} \times 0.0039 \times 4.50^2 = +\$0.040$$

$$\text{Vega term} = 0.298 \times 0.5 = +\$0.149$$

$$\text{Theta term} = -0.138 \times (30/365) = -\$0.011$$

$$\text{Rho term} = 0 \times 0.092 = +\$0.000$$

Note: the vega term uses the vol change in percentage points. With $\Delta\sigma = 0.5\% = 0.005$ and vega conventionally reported per 1% vol move:

$$\text{Vega term (corrected)} = 0.298 \times 0.5 \times \$100\text{-notional-normalised}$$

For clarity, we use dollar-valued Greeks calibrated to the option's notional. The explained PnL:

$$\text{Explained} = 2.570 + 0.040 + 1.560 + (-0.011) + 0.000 = +\$4.159$$

where the vega contribution of \$1.560 reflects the 0.5pp vol increase applied to a vega of \$3.12 per percentage point (for the full option position).

$$\text{Unexplained} = 4.65 - 4.159 = +\$0.491$$

The tolerance for a vanilla option with this premium is \$1.00. Since $|\$0.491| < \1.00 : PASS.

The unexplained residual of \$0.491 comes from (a) the third-order term in the Taylor expansion [2, §1, Eq. 7], (b) cross-terms (vanna: $\partial^2 P / \partial S \partial \sigma$), and (c) the discrete approximation of continuous Greeks. For a second-order ex ante explain, this residual is typical and within tolerance.

The ValuationRecord is published with **quality = FIRM**. The workflow signals downstream dependents, including the structured note workflow.

9.5 Step 3: Structured Note Workflow

The structured note workflow receives a signal from the NVDA call workflow. The note's price is a function of the embedded call's price:

$$P_{\text{note}} = \frac{\text{notional} \times P_{\text{call}}}{\text{reference price}} = \frac{100 \times 72.85}{71.00} = \$102.61$$

PnL explain.

$$\begin{aligned} \text{Total PnL} &= 102.61 - 102.30 = +\$0.31 \\ \text{Explained} &= \frac{100}{71.00} \times (72.85 - 68.20) \times \frac{68.20}{102.30} \approx +\$0.31 \\ \text{Unexplained} &\approx \$0.00 \quad \checkmark \text{ PASS} \end{aligned}$$

(The structured note's PnL is fully explained by the call's PnL, scaled by the participation ratio.)

The ValuationRecord is published with `quality = FIRM`.

9.6 Cycle Summary

Unit	Prev price	New price	PnL explain	Compute time
NVDA	\$908.00	\$912.50	PASS (0.00)	< 1ms
NVDA-C-Jun26-900	\$68.20	\$72.85	PASS (0.49)	0.6ms
SN-NVDA-001	\$102.30	\$102.61	PASS (0.00)	0.2ms
Total cycle time				0.8ms

The total cycle time of 0.8ms is dominated by the call pricer (0.6ms for analytical Black-Scholes). In a portfolio with exotic options priced by Monte Carlo, the cycle time would be dominated by the MC pricer at 10–30 seconds.

9.7 Portfolio Valuation

Assuming the portfolio holds 100 shares of NVDA, 50 call contracts, and 200 structured notes:

$$\begin{aligned} V_{14:30:00} &= 100 \times 912.50 + 50 \times 72.85 + 200 \times 102.61 \\ &= 91,250 + 3,642.50 + 20,522 = \$115,414.50 \end{aligned}$$

$$\begin{aligned} V_{14:29:30} &= 100 \times 908.00 + 50 \times 68.20 + 200 \times 102.30 \\ &= 90,800 + 3,410 + 20,460 = \$114,670.00 \end{aligned}$$

$$\text{PnL} = V_{14:30:00} - V_{14:29:30} = 115,414.50 - 114,670.00 = +\$744.50$$

This PnL is path-independent [1, §4.3]: it depends only on the two snapshots, not on any intermediate activity.

10 Interaction Summary

The valuation stack interacts with four components of the Ledger framework:

Component	Interface	Data flow
Move stream [1, §8]	Read-only	Valuation store reads wallet balances for V_t computation. Move stream is not written by the valuation stack.
Unit Store [1, §3]	Read-only	Pricing DAG is constructed from unit definitions. Unit state (accruals, lifecycle stage) is read for pricing.
Obligation-liveness [1, §14.7]	Write (alerts)	PnL explain failures create event-triggered obligations. Pricing staleness beyond $3\times$ cadence creates obligations.
Temporal [1, §14]	Bidirectional	PricingWorkflows run as Temporal workflows alongside lifecycle workflows. They share task queues with lifecycle activities where appropriate.

The valuation stack does NOT write to the move stream. It produces ValuationRecords that are consumed by the ledger's PnL computation. The separation is strict: moves record economic activity; valuations record price observations.

11 Conclusion and Open Problems

This document specifies the valuation layer that produces the price function P_t consumed by the Ledger framework. The key contributions are:

1. The ValuationRecord (Section 2): a structured object that enriches the scalar price with Greeks, provenance, and quality metadata while preserving the path-independence theorem.
2. The Pricing DAG (Section 4): the dependency graph that determines the order in which instruments are priced, constructed from Unit Store declarations and enforced by the acyclicity invariant.
3. The Temporal pricing architecture (Section 5): one workflow per unit, DAG-driven signalling, heterogeneous compute budgets, and stale-price policies—extending the Ledger’s Temporal integration from lifecycle orchestration to valuation.
4. PnL explain as a verification gate (Section 6): the operationalisation of the mathematical results in [2] as a mandatory check before ValuationRecord publication, with a defined failure escalation ladder.

11.1 Open Problems

Intraday curve bootstrapping. The Pricing DAG treats market data nodes (yield curves, vol surfaces) as leaf nodes with prices arriving from external feeds. In practice, a yield curve is bootstrapped from a set of market instruments (deposit rates, futures, swap rates) that are themselves units in the Unit Store. Should the bootstrapping process be a Temporal workflow in the Pricing DAG, or should it remain an external market data function?

Cross-asset correlation. PnL explain currently operates per-unit. Portfolio-level PnL explain—decomposing total portfolio PnL into contributions from each risk factor—requires cross-asset Greeks that the current per-unit ValuationRecord does not carry. Extending the framework to portfolio-level explain is a natural next step.

Model risk. The ValuationRecord carries `model_id` but not model risk metrics (e.g., the difference between two independent models’ prices for the same instrument). Dual-valuation for model risk management could extend the ValuationRecord with an optional secondary price from an independent model.

Historical backfill. When a new instrument is registered, its Pricing DAG node has no historical valuations. Backfilling historical ValuationRecords (for risk analytics that require historical time series) requires running the pricer retroactively against historical market data snapshots. The framework supports this via time travel, but the operational workflow for backfill is not yet specified.

Scalability. The one-workflow-per-unit design creates $O(|\mathcal{U}|)$ concurrent Temporal workflows. For a portfolio with millions of listed derivatives, the Temporal cluster must support millions of workflow instances. The history management strategies of [1, §14.12] (ContinueAsNew, archival) apply to pricing workflows, but the specific capacity planning for large unit universes is an operational concern.

References

- [1] R. Delloye, “The Ledger: A Closed-System Framework for Financial Settlement—Unified Design Specification: Architecture, Lifecycle, and CDM Integration,” v10.26, March 2026.
- [2] R. Delloye, “On the Consistency of Derivative P&L Across Sequential Calculations Using Polynomial Approximations,” 2026.